

Contents

1 Cargar datos después de la creación de tablas en Spring Boot	1
1.1 Solución 1: Usar 'spring.jpa.defer-datasource-initialization=true' y 'data.sql'	1
1.2 Solución 2: Usar 'schema.sql' y 'data.sql' por separado	2
1.3 Solución 3: Usar 'CommandLineRunner' para cargar datos después de la creación de las tablas	4

1 Cargar datos después de la creación de tablas en Spring Boot

Para cargar los datos **después de la creación de las tablas** y evitar errores en Spring Boot, es importante que la carga de los datos en el archivo '**data.sql**' ocurra **después** de que las tablas hayan sido creadas. Esto se puede lograr ajustando la configuración de Spring Boot y el orden de inicialización de los datos.

1.1 Solución 1: Usar 'spring.jpa.defer-datasource-initialization=true' y 'data.sql'

Este es el enfoque más sencillo y recomendado en la mayoría de los casos. Puedes configurar '**spring.jpa.defer-datasource-initialization=true**' para que las tablas sean creadas automáticamente antes de que se carguen los datos del archivo '**data.sql**'.

1. Asegurarte de que 'spring.datasource.initialization-mode' esté correctamente configurado

Debes establecer la propiedad '**spring.datasource.initialization-mode=always**' en '**application.properties**' o '**application.yml**' para garantizar que los scripts de SQL (como '**data.sql**') se ejecuten después de la creación de las tablas.

Ejemplo 'application.properties':

```
# Configuración de la base de datos
spring.datasource.url=jdbc:postgresql://localhost:5432/biblioteca_db
spring.datasource.username=tu_usuario
spring.datasource.password=tu_contraseña
spring.datasource.driver-class-name=org.postgresql.Driver

# Crear o actualizar las tablas automáticamente con Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
# Diferir la creación de tablas
spring.jpa.defer-datasource-initialization=true
# Garantizar que los scripts SQL se ejecuten siempre
spring.datasource.initialization-mode=always
```

Flujo de trabajo:

1. **Hibernate** primero **creará o actualizará** las tablas según las entidades definidas en el proyecto.
2. **Spring Boot** ejecutará el archivo '**data.sql**' automáticamente **después** de que las tablas hayan sido creadas.

1.2 Solución 2: Usar 'schema.sql' y 'data.sql' por separado

Si deseas tener más control sobre cuándo se crean las tablas y cuándo se cargan los datos, puedes usar un archivo 'schema.sql' para definir el esquema de la base de datos (crear las tablas manualmente) y un archivo 'data.sql' para insertar los datos **después** de la creación de las tablas.

Crea 'schema.sql' para definir el esquema de la base de datos

Este archivo se encarga de crear las tablas en la base de datos. Colócalo en 'src/main/resources/'.

'schema.sql':

```
-- Crear tabla autor
CREATE TABLE IF NOT EXISTS autor (
    id BIGINT PRIMARY KEY,
    nombre VARCHAR(255),
    nacionalidad VARCHAR(255)
);

-- Crear tabla libro
CREATE TABLE IF NOT EXISTS libro (
    id BIGINT PRIMARY KEY,
    titulo VARCHAR(255),
    genero VARCHAR(255),
    año_publicacion INT,
    estado VARCHAR(50),
    autor_id BIGINT,
    FOREIGN KEY (autor_id) REFERENCES autor(id)
);

-- Crear tabla usuario
CREATE TABLE IF NOT EXISTS usuario (
    id BIGINT PRIMARY KEY,
    nombre VARCHAR(255),
    email VARCHAR(255),
    password VARCHAR(255),
    rol VARCHAR(50)
);

-- Crear tabla prestamo
CREATE TABLE IF NOT EXISTS prestamo (
    id BIGINT PRIMARY KEY,
    libro_id BIGINT,
    usuario_id BIGINT,
    fecha_prestamo DATE,
    fecha_devolucion DATE,
    FOREIGN KEY (libro_id) REFERENCES libro(id),
    FOREIGN KEY (usuario_id) REFERENCES usuario(id)
```

```
);
```

2. Crea **'data.sql'** para insertar los datos Este archivo insertará los datos una vez que las tablas han sido creadas. Colócalo en **'src/main/resources/'**.

'data.sql':

```
-- Insertando autores
INSERT INTO autor (id, nombre, nacionalidad)
VALUES (1, 'Gabriel García Márquez', 'Colombiana');
INSERT INTO autor (id, nombre, nacionalidad)
VALUES (2, 'Isabel Allende', 'Chilena');

-- Insertando libros
INSERT INTO libro (id, titulo, genero, año_publicacion, estado, autor_id)
VALUES
(1, 'Cien años de soledad', 'Novela', 1967, 'disponible', 1),
(2, 'El amor en los tiempos del cólera', 'Novela', 1985, 'disponible', 1),
(3, 'La casa de los espíritus', 'Novela', 1982, 'disponible', 2);

-- Insertando usuarios
INSERT INTO usuario (id, nombre, email, password, rol) VALUES
(1, 'Usuario1', 'usuario1@example.com', '123456', 'usuario'),
(2, 'Bibliotecario1', 'bibliotecario1@example.com', '123456', 'bibliotecario');

-- Insertando préstamos
INSERT INTO prestamo (id, libro_id, usuario_id, fecha_prestamo,
fecha_devolucion)
VALUES
(1, 1, 1, '2024-01-10', NULL),
(2, 2, 1, '2024-01-12', NULL);
```

3. Configurar **'application.properties'**

En este caso, no necesitas que Hibernate cree o actualice las tablas, porque las crearás tú mismo con **'schema.sql'**. Por lo tanto, cambia **'spring.jpa.hibernate.ddl-auto'** a **'none'**.

Ejemplo **'application.properties':**

```
# Configuración de la base de datos
spring.datasource.url=jdbc:postgresql://localhost:5432/biblioteca_db
spring.datasource.username=tu_usuario
spring.datasource.password=tu_contraseña
spring.datasource.driver-class-name=org.postgresql.Driver

# No usar Hibernate para crear tablas, ya que usamos schema.sql
spring.jpa.hibernate.ddl-auto=none

# Garantizar que los scripts SQL se ejecuten siempre
```

```
spring.datasource.initialization-mode=always
```

Explicación del flujo:

1. El archivo 'schema.sql' será ejecutado por Spring Boot para **crear las tablas** antes de que cualquier otra operación ocurra.
2. Una vez que las tablas han sido creadas, 'data.sql' se ejecutará para insertar los datos iniciales en la base de datos.

1.3 Solución 3: Usar 'CommandLineRunner' para cargar datos después de la creación de las tablas

Si prefieres cargar los datos mediante código y no con SQL plano, puedes usar 'CommandLineRunner' para insertar los datos una vez que la aplicación haya arrancado completamente y las tablas estén listas.

Ejemplo 'DataLoader.java':

```
package com.biblioteca.gestion.config;

import com.biblioteca.gestion.entities.Autor;
import com.biblioteca.gestion.entities.Libro;
import com.biblioteca.gestion.repositories.AutorRepository;
import com.biblioteca.gestion.repositories.LibroRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class DataLoader implements CommandLineRunner {

    @Autowired
    private AutorRepository autorRepository;

    @Autowired
    private LibroRepository libroRepository;

    @Override
    public void run(String... args) throws Exception {
        // Crear autores
        Autor autor1 = new Autor(null, "Gabriel García Márquez",
                                "Colombiana", null);
        Autor autor2 = new Autor(null, "Isabel Allende", "Chilena", null);
        autorRepository.save(autor1);
        autorRepository.save(autor2);

        // Crear libros
    }
}
```

```

    Libro libro1 = new Libro(null,
                             "Cien años de soledad", "Novela", 1967,
                             "disponible", autor1);
    Libro libro2 = new Libro(null, "El amor en los tiempos del cólera",
                             "Novela", 1985, "disponible", autor1);
    Libro libro3 = new Libro(null, "La casa de los espíritus", "Novela",
                             1982, "disponible", autor2);
    libroRepository.save(libro1);
    libroRepository.save(libro2);
    libroRepository.save(libro3);
}
}

```

Ventaja de 'CommandLineRunner':

- Se ejecuta **después** de que las tablas han sido creadas y Hibernate ha terminado la configuración.
- Te permite realizar una lógica más compleja para cargar datos iniciales.

Conclusión Para evitar errores al cargar los datos después de la creación de las tablas:

1. **Solución 1:** Usa '**data.sql**' junto con '**spring.jpa.hibernate.ddl-auto=update**' para que Hibernate se encargue de crear las tablas antes de insertar los datos.
2. **Solución 2:** Usa '**schema.sql**' y '**data.sql**' por separado, para tener control manual sobre la creación de tablas y la inserción de datos.
3. **Solución 3:** Usa un '**CommandLineRunner**' para insertar los datos mediante código, una vez que la aplicación esté completamente arrancada.

Cualquiera de estos enfoques asegurará que los datos se inserten correctamente después de la creación de las tablas.